FAITHDROID: Automatically Measurable Explanation Faithfulness for Cost-Aware LLM-Based Android Malware Detection under Concept Drift

Project Report

MSc Practicum Internship Part 2
Cybersecurity

Vinay Appajigowda
Student ID: 24262013

School of Computing
National College of Ireland

Supervisor: Michael Pantridge

August 2026

FAITHDROID: Automatically Measurable Explanation Faithfulness for Cost-Aware LLM-Based Android Malware Detection under Concept Drift

Vinay Appajigowda
24262013

Abstract

Android malware detection appears close to solved, but the accuracies supporting that impression are obtained on random train/test splits that conceal temporal concept drift. Large Language Models (LLMs) now produce readable, behaviour-level accounts of why an application is malicious, yet their faithfulness is almost never measured automatically and the cost of reasoning is paid on every input. This study asks whether conformal-calibrated selective escalation to an LLM reasoner can preserve detection quality under drift while reducing LLM usage and measuring explanation faithfulness without human ground truth. FAITHDROID combines a gradient-boosted detector over 475 hybrid static and dynamic features, an uncertainty gate, Mondrian conformal abstention, retrieval of labelled neighbours from the training period, and an audit that constrains the reasoner to a closed artifact vocabulary. Evaluation uses 76,712 timestamped KronoDroid applications across three temporal windows pinned to 10% malware prevalence. Margin-based routing concentrates detector errors 13.07–14.42× where conformal novelty reaches only 0.47–1.37; selective escalation improves F1 on two of three windows (+6.51, $p = 0.0001$; +2.30, $p = 0.0167$) while accepting every LLM verdict degrades F1 by 15.56. Removing the closed vocabulary collapses grounding from 0.991 to 0.000. The contribution is an annotation-free faithfulness audit; its limitation is that grounding measures integrity, not correctness, since adversarially fooled verdicts scored 1.000 grounding.

Keywords: Android malware detection; concept drift; large language models; conformal prediction; explanation faithfulness; adversarial robustness.

1 Introduction

Android is the dominant mobile platform and the dominant target for mobile malware. By the conventional measure the detection problem looks close to solved: recent work reports 98.2% accuracy (Rashid et al.; 2025), 99.03–100% with F1 reaching 1.00 across four datasets (Saeed et al.; 2026), and 97.82% detection with 96.09% categorisation (Alhogail and Alharbi; 2025). These figures are not evidence that the problem is solved; they are evidence that the standard evaluation protocol has been exhausted. All are obtained on random train/test splits, in which training and test data are drawn from the same period. Malware evolves after a detector is deployed, and across a temporal

boundary performance falls sharply — a Drebin-style detector decays from 81.33 to 61.59 F1 across three successive windows (Qian et al.; 2025), and drift in Android detection is now documented empirically (Minnei et al.; 2024).

Two further requirements have moved to the centre of the problem. The first is *explainability*: a verdict is of limited use to an analyst without an account of which behaviours justify it. The second is *cost*. LLMs are applied to malware analysis because they produce readable, behaviour-level accounts and generalise beyond their training data, but they are expensive: recent frameworks cost $0.199 (Qian et al.; 2025) and $6.00 (Zhang et al.; 2025) per application, and in the latter case the authors state that this expense prevented evaluation at scale.

1.1 Problem statement and research question

The central problem is that **explanation faithfulness in LLM-based Android malware analysis is asserted rather than measured, and the cost of obtaining an explanation is paid on every input regardless of whether it is needed**. Evidence for the first half is direct: TraceRAG presents itself as explainable, yet its own expert study over 24 applications found reviewers confirmed only 53.33% of classifications (Zhang et al.; 2025); in RAG-Android the LLM description is consumed as an input feature, so a hallucinated but discriminative description scores as well as a truthful one (Sakthidharan et al.; 2026); a systematic review finds explanation quality validated by user study in only 14% of the security literature (Nadeem et al.; 2023); and a survey states plainly that evaluating faithfulness remains open (Manthena et al.; 2025).

> *Can conformal-calibrated selective escalation to an LLM reasoner, operating over a hybrid static-and-dynamic Android evidence bundle, preserve detection performance under concept drift while reducing LLM usage and automatically measuring explanation faithfulness without requiring human ground truth?*

Five objectives make this measurable. **O1**: quantify how far conventional detectors degrade under temporal drift. **O2**: determine whether uncertainty-based routing concentrates detector errors while reducing LLM calls. **O3**: establish whether the LLM stage improves the final decision when used selectively rather than uniformly. **O4**: define and validate automatic faithfulness metrics based on artifact grounding and counterfactual testing. **O5**: characterise cost, abstention, faithfulness and adversarial robustness across the cascade.

1.2 Contributions and structure

This work contributes (i) three annotation-free faithfulness metrics — Evidence Grounding Rate, Counterfactual Necessity and Attribution Agreement; (ii) a bridge between the explainable-AI and LLM literatures, auditing narratives with SHAP machinery; (iii) uncertainty-gated selective escalation reported as a cost–accuracy curve; (iv) two negative results of practical value, namely that conformal novelty is no better than random for *ranking* samples and that accepting every LLM verdict degrades F1 by 15.56 points; and (v) reproducibility at zero API cost on one consumer machine. Two limitations are stated at the outset: results come from a single dataset ending in 2020, and the metrics measure explanation *integrity* rather than correctness, as Section 6 demonstrates under adversarial conditions. Section 2 reviews the literature and establishes the gap; Section 3

sets out the method; Section 4 the architecture; Section 5 the implementation; Section 6 the results; Section 7 discusses them; Section 8 concludes.

2 Related Work

Relevant work falls into five strands that have developed largely independently. Except where marked as a preprint, every work reviewed is peer reviewed.

2.1 LLM-based Android malware analysis

TraceRAG (Zhang et al.; 2025) decompiles an APK, summarises method-level units with an LLM and answers behaviour queries through multi-turn retrieval, reporting 96.00% accuracy. It is an arXiv preprint, so its figures are claims rather than established results. Three limitations matter: the evaluation set is only 100 applications, which the authors attribute to budget at roughly **$6.00 per application**; there is no machine learning baseline and no temporal split; and explanation quality rests on an expert study in which Google Android Security specialists across 24 APKs confirmed **only 53.33% of classifications**. That evaluation needs scarce expert labour and cannot be repeated at scale.

LAMD (Qian et al.; 2025) is the baseline this work extends and is methodologically the strongest of the group. It isolates security-sensitive APIs by inter-procedural backward slicing, reasons in tiers from function behaviour to an APK-level verdict, and mitigates hallucination with a factual-consistency check. It is evaluated on AndroZoo across three time-aware windows spanning 2020–2023, reporting roughly 90 F1 throughout while Drebin (Arp et al.; 2014) decays from 81.33 to 61.59. Its contribution is to show that tiered LLM reasoning resists drift. What it does not solve is threefold: drift resilience costs a false positive rate about **three times Drebin's**, with no mechanism to abstain; the system is static-only; and it costs **$0.199 per APK paid on every input**. Critically its explanations are never evaluated for faithfulness — the consistency check validates an intermediate summary's dependency structure, not whether the final explanation is grounded in the application. The paper names the gap FAITHDROID addresses, calling for "effective strategies to combine these two paradigms".

RAG-Android (Sakthidharan et al.; 2026) generates descriptions of manifest features through an agentic RAG pipeline and classifies the text with CySecBERT for 93.31% accuracy. Because the description is consumed purely as an input feature and nothing verifies it, **hallucination is structurally invisible to the metric**. Two 2025 systems raise capability without closing that gap: Zhao et al. (2025) drive detection via multi-view prompt engineering and Feng et al. (2025) report a full LLM-based detector, while Jing et al. (2026) derive security-aware behavioural embeddings that are again consumed as feature vectors. Rollinson and Polatidis (2026) report that real data alone already achieves near-perfect detection, establishing KronoDroid (Guerra-Manzanares et al.; 2021) as current and evidencing benchmark saturation.

2.2 Classical detectors and explainable AI for malware

Rashid et al. (2025), Saeed et al. (2026) and Alhogail and Alharbi (2025) report 97–100% accuracy from hybrid feature sets. All three assert explainability by construction — a

feature ranking, a two-stage decomposition, a mapping from outputs to patterns — and never validate it, and none uses a temporal split.

Three sources make the measurement gap explicit. Saqib et al. (2024), in ACM Computing Surveys, warn that explainable detectors are *more* vulnerable to attack, and their own contribution assigns qualitative *levels* of explainability precisely because no quantitative measure exists. Manthena et al. (2025) state that evaluating faithfulness remains open while covering no LLM-based method. Nadeem et al. (2023) find user-study validation in only 14% of cases, warn that qualitative analysis invites cherry-picking, and observe that most explanation criteria can be evaluated without humans and that fidelity-by-perturbation is established precedent — two observations that shape Section 4. Empirically, Galadima et al. (2026) compare LEMNA, LIME and SHAP and find **low Jaccard agreement between explainers**, concluding against reliance on any single one; Najibi and Bidgoly (2025) and Singh et al. (2024) apply attribution to Android detection but stop at attribution rather than verification.

2.3 Drift, conformal prediction, cost and adversarial robustness

Barbero et al. (2022) use conformal evaluation to *reject* low-credibility predictions under drift, and Zhang et al. (2021) apply inductive conformal learning to unseen Android malware, using the signal to select data for updates. In both, conformal machinery governs *which samples enter training*; this work instead uses calibrated uncertainty to decide *which samples escalate at inference*. Li et al. (2025) show adaptation to genuinely drifted malware from minimal samples, and He et al. (2025) — from the LAMD group — couple drift detection to *explanation*, using explanatory signals to adapt. That is the closest work in spirit, but it uses explanation to improve classification whereas this work audits whether the explanation is itself grounded. Sabbah et al. (2025), a preprint, find few-shot LLM prompting did not fully mitigate drift.

On cost, Kwon et al. (2026) treat drift-aware maintenance as an operational decision problem, matching cumulative retraining while reducing update volume; they economise *retraining* cost whereas this work economises the cost of *reasoning about an individual sample*. Augello et al. (2025) is closest on routing, but escalates to dynamic analysis rather than an LLM and has no explainability component. Retrieval grounding is established by Fayyazi et al. (2024), and Shukla and Joshi (2025) argue that retrieval-augmented answers should carry checkable provenance, establishing it at document level; this work instead constrains generation to a finite vocabulary of application artifacts so provenance is checked against the object under analysis. I et al. (2024) treat factual inconsistency as the central obstacle wherever an ungrounded assertion carries cost. Finally, Rafiq et al. (2022) show hybrid-feature Android classifiers are evadable and Lan and Nait-Abdesselam (2025) use an LLM to construct attacks, while Noppel and Wressnegger (2024) systematise attacks against post-hoc explanation methods and conclude they are **not themselves robust to manipulation**.

2.4 The research gap

Table 1 contrasts the closest systems. Three findings converge. **G1: faithfulness is never measured automatically** — no annotation-free metric exists for whether an explanation is grounded in the artifacts an application contains, even though Nadeem et al. (2023) note most such criteria need no humans. **G2: LLM reasoning is spent**

uniformly — $6.00 and $0.199 per application are both paid on 100% of inputs, and while Kwon et al. (2026) show cost-aware selectivity works for retraining, no published Android system applies it to the per-sample reasoning budget. **G3: there is no statistically valid abstention** — Qian et al. (2025) accept a threefold FPR increase with no safe referral path, and although conformal prediction supplies that guarantee (Barbero et al.; 2022; Zhang et al.; 2021), neither couples it to a reasoning stage.

Table 1: Comparison with the closest prior systems. "Temporal" indicates whether concept drift is evaluated; "Faithf." whether explanation quality is quantified rather than asserted.

System	Dataset	Evidence	Explanation	Temp.	Faithf.	Main limitation
TraceRAG (Zhang et al.; 2025)	100 apps	Static (Java)	Narrative	No	Expert only	$6.00/app; no ML baseline; experts confirmed 53.33%
LAMD (Qian et al.; 2025)	AndroZoo	Static slices	Indicators	**Yes**	No	$0.199/app on 100% of inputs; FPR 3× Drebin; no abstention
RAG-Android (Sakthidharan et al.; 2026)	18k apps	Static manifest	Feature text	No	No	Hallucination invisible to the metric
C-STAR (Kwon et al.; 2026)	AndroZoo	Static	None	**Yes**	No	Optimises retraining cost, not per-sample reasoning
FAITHDROID	**KronoDroid**	**Static + dynamic**	**Closed-vocabulary**	**Yes**	**Yes, auto**	**Local 7B; $0 API; LLM on ∼10% of inputs**

3 Research Methodology

3.1 Design and dataset

The study is quantitative and experimental: a system is built, and each objective is answered by a measurement against a fixed protocol rather than by argument. It uses **KronoDroid** (Guerra-Manzanares et al.; 2021), real-device variant, the only suitable dataset in the reviewed literature for three reasons. It is genuinely **hybrid**, carrying system-call counts captured on a real device alongside static permission features, where every LLM-based system in Section 2 is static-only. It is **timestamped**, permitting true time-aware splits, and is openly downloadable where AndroZoo requires a key. And its **feature names are human-readable** (`SEND_SMS`, `execve`) — a precondition, since the faithfulness metrics require a closed, machine-checkable vocabulary that anonymous numeric features could not supply.

The real-device set holds 41,382 malware and 36,755 benign samples across 484 columns and 240 families. Four preparation decisions each respond to a property of the data. Emulator data is **excluded**, since that file lacks exactly the timestamp fields this study depends on. Four SHA-256 duplicates were **removed**, as a repeated application spanning a temporal boundary would leak. `HighestModDate` was **adopted** over `EarliestModDate` (usable for 98.2% of records against 88.5%), discarding records outside [2008-01-01, 2021-06-30] and retaining **76,712** samples; these dates are attacker-

controllable and are a proxy for first-seen date. Features were **grouped** into 288 system-call and 166 permission columns plus 21 counters, 475 in total; the 70 unnamed `SYS_nnn` columns are retained as classifier features but excluded from the evidence bundle, and VirusTotal-derived columns are excluded entirely as label leakage.

3.2 Temporal splits and data roles

Splitting by time naively introduces a confound: per-year class balance swings from 88% benign in 2011 to 93–96% malware in 2012–2014, so a model trained on one period and tested on another faces a prior shift simultaneously with any genuine change in concept. Every window is therefore subsampled to a fixed 10% malware prevalence, matching the 9.42–10.01% of LAMD's windows (Qian et al.; 2025) so the operating regime is comparable (Table 2).

Table 2: Prevalence-controlled temporal splits. Training and calibration both precede every test window, and all five sets are pinned to 10% malware so differences across windows reflect concept drift rather than a change in class prior.

Split	Period	n	Malware	Benign	% malware
Train	≤ 2014	23,581	2,358	21,223	10.00
Calibration	≤ 2014	10,107	1,011	9,096	10.00
Test1	2015–2016	1,627	163	1,464	10.02
Test2	2017–2018	1,591	159	1,432	9.99
Test3	2019–2020	2,848	285	2,563	10.01

The roles are kept strictly separate. **Training** data fits the detector. **Calibration** data fits the conformal predictor and the override band, and nothing else. The three **test** windows are used only for final measurement. Training and calibration are separated by a *stratified random* split: a chronological sub-split was attempted first and proved degenerate, producing 1.01% malware in the earlier half against 30.97% in the later, because benign samples concentrate in 2010–2011 and malware in 2012–2014. No target-window data informs any threshold, so the override policy in Section 4 is genuinely target-independent; where any analysis does use test data it is identified as such.

3.3 Baselines, metrics and validity

Four detectors mirror LAMD's baselines: a **Drebin-style** linear SVM (Arp et al.; 2014), a **DeepDrebin-style** multi-layer perceptron, a **Random Forest** of 300 trees, and **XGBoost** with positive-class weighting matched to 10% prevalence. Each is trained once and evaluated unchanged on all three windows. These are LAMD-*style* baselines on KronoDroid, not reproductions of LAMD's AndroZoo results — the drift gap here reaches six years against roughly three — so absolute values are **not** comparable across studies and no claim of outperforming LAMD is made.

Detection quality is reported as F1, precision, recall, FPR and FNR. FPR and FNR appear throughout because accuracy misleads at 10% prevalence, where predicting "benign" everywhere scores 90%, and because in triage the false positive rate governs analyst workload. Since the LLM runs locally, monetary cost is zero, so cost is reported as escalation rate, wall-clock latency and a token-priced equivalent at LAMD's published rates — an equivalence, not a cost incurred. Significance uses paired bootstrap intervals over

applications (10,000 resamples) and exact McNemar tests, appropriate because both systems see identical inputs and their errors are correlated. Threats to validity are stated in Section 7.

4 Design and Architecture

FAITHDROID is a four-stage cascade (Figure 1): a cheap learned detector decides the confident majority, an uncertainty gate routes only the ambiguous minority to an LLM reasoner, conformal prediction supplies validity-backed abstention, and every explanation is audited automatically before acceptance.

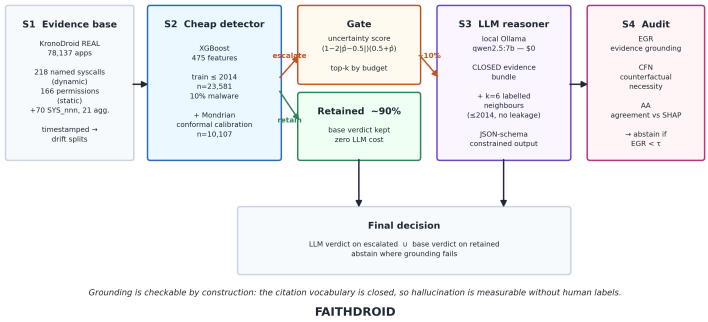


Figure 1: End-to-end FAITHDROID architecture. Static and dynamic evidence are unified into a bundle; the detector resolves roughly 90% of inputs at negligible cost; the reasoner sees only the routed remainder together with labelled neighbours drawn from the training period; and the audit gates the final analyst report.

The decision from which everything else follows is the **closed evidence bundle**. The reasoner is shown a finite, explicitly enumerated set of artifact identifiers taken from the application itself and must cite evidence using only strings from that set. Because the permitted vocabulary is known and finite, any citation can be checked against the application's feature vector mechanically. That is what makes faithfulness measurable at scale, and precisely what prior systems cannot do: TraceRAG needed expert analysts (Zhang et al.; 2025) and RAG-Android renders a hallucinated description indistinguishable from a correct one (Sakthidharan et al.; 2026). It is a stronger guarantee than document-level provenance (Shukla and Joshi; 2025), since verification is against the object under analysis rather than a retrieval corpus.

4.1 Detector, calibration and routing

Each application is represented by 475 features. Only the 218 *named* system calls are bundle-eligible; the anonymous `SYS_nnn` columns remain available to the classifier but are withheld from the reasoner, since they cannot support an analyst-facing explanation. XGBoost is the first-stage detector, wrapped in a **Mondrian (class-conditional) inductive conformal predictor** calibrated on the held-out in-period split. For nonconformity $\alpha(x, y) = 1 - \hat{p}(y \mid x)$, the p-value for label y is

$$p_y = \frac{|\{i \in \mathrm{Cal}_y : \alpha_i \geq \alpha(x, y)\}| + 1}{|\mathrm{Cal}_y| + 1} \tag{1}$$

Class conditioning matters at 10% prevalence, since a pooled calibration set would be dominated by the benign class and the guarantee would not hold per class.

The gate was originally built on these p-values, on the reasoning that a sample merits escalation when it is either ambiguous or novel. That design was tested and rejected on evidence (Section 6). The gate therefore uses a probability-margin score weighted toward the positive class,

$$s(x) = \big(1 - 2|\hat{p}(x) - 0.5|\big) \cdot \big(0.5 + \hat{p}(x)\big) \tag{2}$$

where the first factor is standard uncertainty sampling and the second tilts selection toward suspected positives, because at 10% prevalence false negatives dominate and cost more in triage. Samples are escalated in descending $s(x)$ up to the budget. The conformal predictor is retained for the role in which it is principled — abstention rather than ranking — a division of labour distinguishing this design from Zhang et al. (2021), where the conformal signal instead selects training data.

4.2 Grounded reasoning and the faithfulness audit

Escalated samples pass to a locally hosted open-weight model with output constrained by a JSON schema requiring a verdict, one to six cited artifacts and free-text reasoning. A bare bundle proved insufficient: given permissions alone the model returned MALWARE for 16 of 20 applications in a sample containing one, citing near-universal artifacts such as `INTERNET` and even `VIBRATE` — the surface-cue reliance Zheng et al. (2025) report, reproduced with a measurable mechanism. The remedy supplies the missing reference distribution: for each escalated sample the six most similar labelled applications are retrieved from the *training* period, with similarity computed as Jaccard overlap on binarised artifact presence,

$$\mathrm{sim}(a, b) = \frac{|A \cap B|}{|A \cup B|} = \frac{\mathbf{Mv}}{\|\mathbf{M}\|_1 + \|\mathbf{v}\|_1 - \mathbf{Mv}} \tag{3}$$

which is exact and runs in well under a millisecond. Retrieval draws only from the training split, which ends in 2014, so no evaluation-period label information can leak. This differs from Sakthidharan et al. (2026) in what is retrieved and why: they retrieve *documentation* to generate richer descriptions; this work retrieves *labelled examples* to calibrate a judgement.

Let $\mathcal{A}(x)$ be the closed artifact set and $\mathcal{C}(x)$ the citations. **Evidence Grounding Rate** is $\mathrm{EGR}(x) = |\mathcal{C}(x) \cap \mathcal{A}(x)|/|\mathcal{C}(x)|$, requiring no annotation and detecting both invention and misuse of the citation field. Because a citation can be grounded yet irrelevant, **Counterfactual Necessity** ablates the cited artifacts and re-scores, $\mathrm{CFN}(x) = |\hat{p}(x) - \hat{p}(x \setminus \mathcal{C}(x))|$, reported against a **control** that ablates an equal number of randomly chosen *present* artifacts — without which any movement might reflect removing some features rather than the cited ones — following the fidelity-by-perturbation precedent of Nadeem et al. (2023). **Attribution Agreement** compares citations with the top-k SHAP features against a chance baseline; because Galadima et al. (2026) show explainers disagree, AA is read as corroboration, not ground truth. An explanation below a grounding threshold is not accepted but referred to an analyst.

Finally, the override band $\mathcal{B}$ decides which escalated verdicts replace the detector's own. It is fitted on calibration data by retaining probability bands below a reliability threshold of 0.70, yielding $\mathcal{B} = [0.3, 0.7)$, so no test information informs it. Routing and

override are deliberately separate decisions: a sample can be worth a second opinion while that opinion remains less reliable than the first.

5 Implementation

The system is implemented in Python 3.13 on a single Apple M5 Pro with 24 GB of unified memory. Detection uses scikit-learn 1.5 and XGBoost 3.2, attribution uses SHAP 0.52, and the reasoning stage runs open-weight models locally through Ollama 0.32, so no component depends on a commercial API and the pipeline incurs no per-sample charge. All configuration — paths, split boundaries, target prevalence, gate parameters, model names and thresholds — is centralised in one module, so no experimental parameter is defined in more than one place. Paths are resolved relative to a project root rather than hardcoded, and every random operation is seeded.

Preprocessing loads the two real-device CSVs, aligns them on their shared column set, reports any discrepancy explicitly rather than letting it surface later as missing values, deduplicates on SHA-256, cleans `HighestModDate`, and writes the five splits to Parquet with metadata recording exact feature-group membership. Groups are resolved by position against the `nr_syscalls` and `nr_permissions` delimiters, with assertions that they are non-empty and disjoint, so a change to the dataset layout fails loudly rather than silently mis-classifying features.

The conformal predictor stores calibration nonconformity scores sorted per class, so p-values are computed by binary search rather than by comparing each test point against the whole calibration set. Its validity was verified before the component was used for anything: on the exchangeable calibration split the empirical error of the prediction set was 4.97% at a nominal significance of 0.05 and 9.97% at 0.10.

The neighbour index was first built with scikit-learn's `NearestNeighbors` using the `jaccard` metric, which dominated runtime because the boolean metric is evaluated in Python against all 23,581 training rows per query. Replacing it with the exact vectorised formulation of Equation 3 reduced the cost to **0.81 ms per query**, three orders of magnitude faster, with identical results.

Requests go to Ollama's generate endpoint with a JSON schema in the `format` field, so decoding is constrained to a verdict from a two-value enumeration, one to six citations and a reasoning string; temperature is fixed at zero. Citations are capped at 64 characters, added in response to a failure the faithfulness metric itself discovered when the model inserted explanatory sentences into the citation array; the uncapped configuration is retained as an ablation. Responses are cached by model, prompt mode and a hash of the sample identifier combined with the full prompt text, so any change to prompt construction correctly invalidates the entry and an interrupted sweep resumes without repeating work.

EGR is computed during the LLM stage where the closed artifact set is available; CFN and AA afterwards against the detector. For CFN only *grounded* citations can be ablated, since a hallucinated artifact has no column. For AA the SHAP ranking is restricted to artifacts both present and bundle-eligible, since the unrestricted top-k would include columns the model never saw. Because escalation is a deterministic ranking, the set escalated at a small budget is exactly a prefix of that at a larger one, so one sweep at the largest budget yields every smaller operating point by truncation, removing roughly 800 redundant LLM calls. Complete run order, package versions and regeneration commands

are given in the configuration manual.

Table 3: Learned baselines under temporal drift, reported as F1 / FPR / FNR (%) in the format used by Qian et al. (2025). Each model is trained once on data up to 2014 and evaluated unchanged on all three windows. These are LAMD-*style* baselines on KronoDroid, not reproductions of LAMD's AndroZoo results; absolute values are not comparable across the two datasets.

Model	**Test1 (2015–16)**	**Test2 (2017–18)**	**Test3 (2019–20)**
	F1 / FPR / FNR	F1 / FPR / FNR	F1 / FPR / FNR
Drebin-style linear SVM	48.90 / 16.94 / 18.40	55.17 / 15.29 / 9.43	49.77 / 14.79 / 22.81
DeepDrebin-style MLP	57.43 / 3.62 / 46.63	75.44 / 3.77 / 18.87	62.27 / 1.91 / 47.02
Random Forest	59.52 / 0.96 / 53.99	76.98 / 0.84 / 32.70	17.34 / 0.39 / 90.18
XGBoost (adopted as Stage 2)	66.12 / 5.60 / 25.77	81.56 / 3.70 / 8.18	56.23 / 4.64 / 44.56

Table 4: Cascade performance under the two override rules. *Blind* replaces the base verdict on every escalated sample; *banded* replaces it only where the base detector is known from calibration data to be unreliable ($\hat{p} \in [0.3, 0.7)$). The final two columns are the key diagnostic — accuracy of each stage restricted to the escalated subset — and explain the difference: escalation pays only while the LLM is more accurate than the detector it replaces.

		Blind override			**Banded override**			**Acc. on esc.**	
(lr)3-5(lr)6-8(lr)9-10 **Window**	**base F1**	esc.	F1	Δ	ovr.	F1	Δ	base	LLM
Test1	66.12	10.0%	75.88	+9.76	5.0%	**72.63**	**+6.51**	50.9%	76.7%
Test2	81.56	10.0%	82.76	+1.19	3.1%	**83.09**	**+1.53**	74.8%	78.6%
Test3	56.23	20.0%	40.67	-15.56	3.4%	**58.53**	**+2.30**	68.9%	62.5%

Table 5: Statistical significance of the cascade's gain over the base detector. Intervals are paired percentile bootstraps over applications (10,000 resamples); the test is exact McNemar on discordant pairs, which is the correct choice because both systems are evaluated on identical inputs and their errors are therefore correlated. The gain on Test2 is *not* significant — its interval includes zero — so no improvement is claimed on that window.

Window	**base F1**	**cascade F1**	**gain**	**95% CI**	**discordant**	p
Test1	66.12	72.63	+6.51	$[+3.22, +9.87]$	42	0.0001
Test2	81.56	83.09	+1.53	$[-0.65, +3.85]$	23	0.2100
Test3	56.23	58.53	+2.30	$[+0.77, +3.91]$	35	0.0167

6 Evaluation and Results

All tables and figures in this section are generated directly from the saved experiment output, so the reported figures and the runs that produced them cannot diverge.

6.1 RQ1: how far do detectors degrade under drift?

Table 3 reports the four baselines across the three windows. Where related work reports 98.2% (Rashid et al.; 2025) and 99.03–100% accuracy (Saeed et al.; 2026) on random

Table 6: Automatic faithfulness audit and ablations, on Test3 (2019–20). *Strict* EGR requires each citation to match an artifact exactly; *lenient* EGR first splits citations that pack several names into one field, so the gap between the two isolates output-format violations from genuine invention. CFN lift and AA lift are relative to matched random controls, so 1.00 means indistinguishable from chance. **The first three rows form a chain, not independent removals**: indentation shows the row each is a controlled single-variable comparison against. Removing retrieval also removes the neighbour context, so the copy-exactly rule is isolated by comparing row 3 with row 2, not with row 1. Row 4 removes only the output-schema length cap from the full system.

		Grounding				**CFN**	**AA@10**	
(lr)3-6(lr)7-7(lr)8-9 **Configuration**	**n**	strict	lenient	fmt. viol.	fully	lift	prec.	lift
FAITHDROID (full)	570	0.974	0.980	0.0%	93.7%	1.32	0.366	1.33
− retrieval context	200	0.991	0.994	0.2%	98.5%	1.62	0.420	1.50
− copy-exactly rule	199	0.000	0.044	39.9%	0.0%	0.00	0.000	0.00
− citation length cap *(from full)*	200	0.965	0.974	0.0%	93.0%	1.22	0.377	1.47

splits, the best model here reaches 81.56 F1 on the easiest window and **56.23** on the hardest. Decay is neither monotonic nor uniform: the Random Forest collapses on Test3 from 76.98 to **17.34** F1, classifying almost every 2019–20 application as benign, while XGBoost degrades but survives and is therefore adopted as the first stage. That two models sharing a feature space diverge this sharply shows drift robustness is a property of the decision boundary, not of the features. The Drebin-style linear model fails in the opposite direction, holding recall but at a false positive rate that would be unusable in triage — roughly one false alarm per six applications at 10% prevalence. This answers RQ1 and establishes the headroom a second stage could recover.

6.2 RQ2: can uncertainty routing concentrate errors?

A routing signal is useful only if the samples it selects are disproportionately those the detector gets wrong. Measured by error-concentration lift, **every conformal-derived signal failed**: conformal novelty reached only 0.47–1.37, at or below the 1.0 line marking random selection, meaning escalated samples were sometimes classified *more* accurately than retained ones. The positive-weighted probability margin achieved **13.07–14.42**, and is stable across budgets.

Conformal validity itself was verified before the predictor was relied on (Figure 2): on the exchangeable calibration split the empirical error of the prediction set is near-exact, and it degrades under drift exactly as exchangeability fails.

The diagnosis is visible in the conformal statistics: under drift 36–45% of test samples fall in the empty-prediction-set region, so the signal saturates and ranking within a mass of near-zero p-values approaches random ordering. This does not contradict Barbero et al. (2022), whose use of conformal credibility is to *reject* low-credibility predictions — a threshold decision for which a saturated signal remains informative. The lesson is that the same quantity does not transfer from thresholding to *ranking*. Reported as a negative result because conformal novelty is the field's default tool, it answers RQ2 affirmatively but only for the margin-based signal (Figure 3).

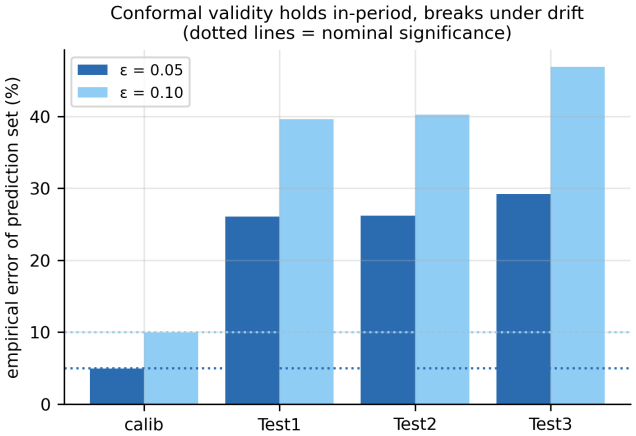


Figure 2: Empirical error of the conformal prediction set against nominal significance. Validity holds almost exactly in-period and degrades under drift, which is the expected consequence of concept drift violating exchangeability. This is why the predictor is confined to abstention rather than ranking.

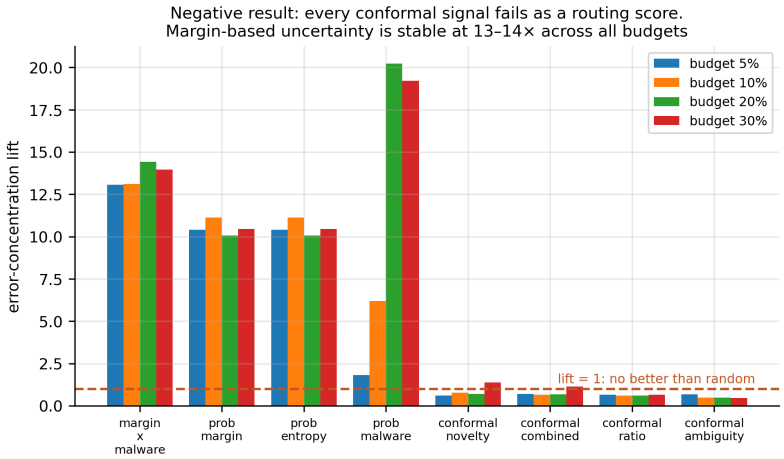


Figure 3: Error-concentration lift by routing signal, averaged across the three windows. The dashed line at 1.0 marks random selection: all conformal signals sit at or below it, while probability-margin uncertainty exceeds it by roughly an order of magnitude.

6.3 RQ3: does selective escalation beat uniform escalation?

Table 4 reports the cascade at a 10% budget and Table 5 the significance tests. On Test1 the base detector is correct on only half of the escalated samples — the gate is finding genuinely hard cases — while the LLM is substantially better on those same samples.

On Test3, however, replacing the base verdict on *every* escalated sample **degrades F1 by 15.56 points**, monotonically in the budget. As the budget expands the escalated set stops being composed of hard cases, base accuracy on it rises, and past the crossover the system replaces a better predictor with a worse one; the errors introduced are specifically missed malware, because the nearest labelled neighbours available to a 2019–20 application are drawn from 2014 and earlier and are predominantly benign.

The remedy follows from the diagnosis: override only where the detector is *known* to be unreliable, a region identifiable from calibration data alone. The banded rule converts that -15.56 into $+\mathbf{2.30}$, stable across budgets because the override rate saturates at 3.4% of inputs. Under paired bootstrap and exact McNemar, Test1 gains $+6.51$ ($p = 0.0001$) and Test3 $+2.30$ ($p = 0.0167$), both significant, while **Test2 gains $+1.53$ at $p = 0.21$ — not significant**. The honest answer to RQ3 is therefore that the cascade demonstrably improves two of three windows and is indistinguishable from the detector on the third; and that selective escalation is essential, since uniform acceptance is actively harmful.

Figure 4 plots the resulting cost–accuracy trade-off, on which prior LLM-based systems sit at an escalation budget of 100% — far to the right of any point plotted here.

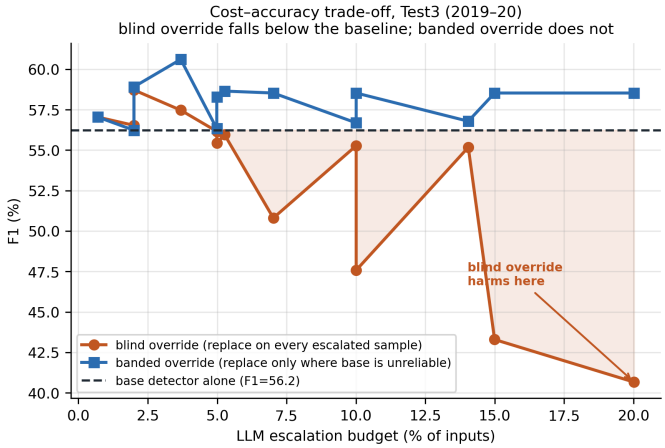


Figure 4: Cost–accuracy trade-off on Test3 (2019–20). The shaded region marks where blind override performs *worse* than the cheap detector alone; the banded rule stays above the baseline across the whole budget range.

6.4 RQ4: can faithfulness be measured automatically?

Table 6 reports the three metrics with three ablations. The decisive result is that **the grounding constraint is entirely load-bearing**: removing the instruction to copy citations verbatim drops mean EGR from **0.991 to 0.000**. Not one of 562 citations across 199 applications referred to an artifact the application contained, and 495 of them contained spaces — the model emits prose where evidence is expected. CFN and AA become undefined for that configuration, which is itself the point: the closed vocabulary is not an incremental improvement but the precondition for explanation quality being

measurable at all. This is exactly the condition Sakthidharan et al. (2026) operate in without measuring it.

Retrieval creates a genuine tension. Removing labelled-neighbour context *improves* every faithfulness metric — EGR rises from 0.974 to 0.991 — while badly damaging verdict accuracy, because the model cites ubiquitous rather than discriminative artifacts. Grounding and necessity are therefore distinct properties, as Figure 5 shows across all four configurations.

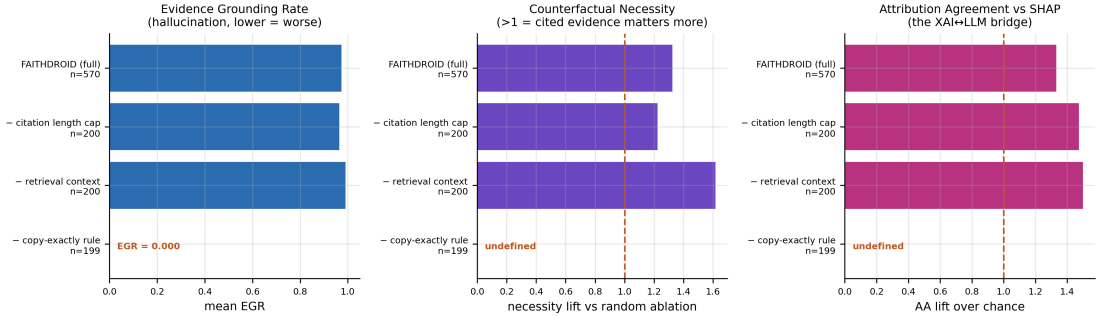


Figure 5: Automatic faithfulness audit across configurations. EGR measures grounding; CFN necessity is reported against a matched random-ablation control and AA against a chance baseline, so for the latter two a value of 1.0 denotes indistinguishable from chance.

A validity check on EGR confirms it measures real invention: of 106 ungrounded citations only 2 (1.9%) name an artifact the application actually contains but which fell outside the displayed bundle. RQ4 is answered affirmatively.

6.5 RQ5: cost, scale, abstention and adversarial robustness

Verdict accuracy is flat across an almost fivefold parameter range — 63.3% at 3B, 62.5% at 7B, 64.8% at 14B — so the benefit comes from the architecture rather than model scale, which is what makes a zero-cost local deployment viable against the $600 and $1,800 experimental costs of the closest prior work. Raising the grounding threshold declines 6% of explanations and reduces error among those retained only slightly (Figure 6), which is the expected behaviour of a metric measuring support rather than correctness: abstaining on ungrounded explanations is justified because an unsupported explanation should not reach an analyst, not because it materially improves accuracy.

The adversarial evaluation is the most consequential result. Under a feature-addition threat model — the attacker may add benign-looking artifacts but never remove any, so the malware keeps working — five added permissions evade the detector on **28.1%** of previously-caught Test1 malware and **50.0%** of Test3 malware. Drift and adversarial vulnerability compound: the window that drift has already weakened is also the cheaper one to attack. Passing evaded samples through the reasoner, the LLM was also fooled on 61.8% of Test1 and 97.4% of Test3 evasions, and on *every* sample where both stages were fooled mean grounding was **1.000** — every cited artifact genuinely present, every verdict wrong. Worse, the relationship runs the wrong way: grounding was *higher* when the system was fooled (1.000) than when it resisted (0.782). On Test1, 55.9% of explanations cited an artifact the *attacker had injected*. RQ5 is therefore answered, but negatively on robustness: the explanation channel is itself an attack surface (Figure 7).

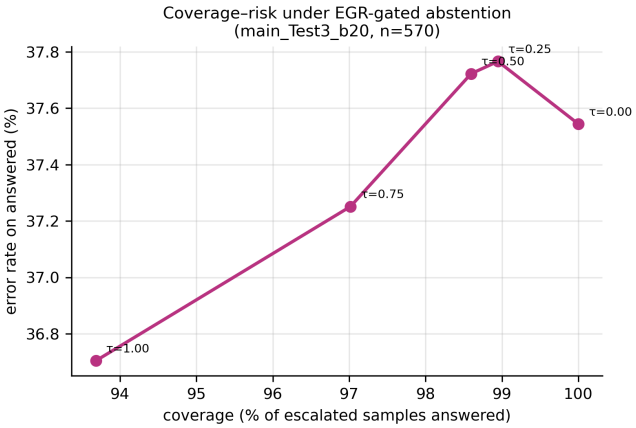


Figure 6: Coverage–risk trade-off when explanations below a grounding threshold are declined rather than shown. The effect is real but small, which is consistent with grounding and correctness being distinct properties.

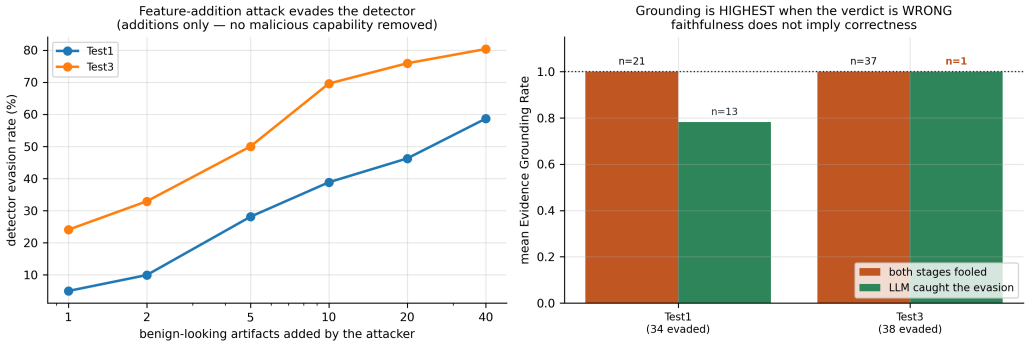


Figure 7: Left: a feature-addition attack evades the detector at modest budgets, far more easily on the most drifted window. Right: mean grounding when both stages were fooled versus when the reasoner resisted — grounding is at its maximum on wrong verdicts.

6.6 Threats to validity

Four threats bound what these results establish, and each is stated here rather than left for a reader to infer.

Construct validity. `HighestModDate` is derived from APK-internal file modification dates, which an application author can forge. It is a proxy for first-seen date, not a guarantee of it. Cleaning removed the obvious artefacts — the ZIP epoch and impossible future dates — but plausible-looking forgery is undetectable, so the temporal ordering on which every drift claim rests is only as reliable as that field.

Internal validity. The override band is fitted on calibration data that strictly precedes every test window, so no evaluation-period information informs the policy and the comparison is honest. One asymmetry remains: calibration identifies the high-probability region as reliable, yet under drift its test accuracy falls well below the calibration estimate. Calibration cannot anticipate that degradation, so the rule leaves a region of genuine unreliability unaddressed — a limitation of the policy, not of the measurement.

External validity. Results come from KronoDroid alone, which ends in 2020. The prevalence-controlled temporal protocol transfers to any timestamped corpus, but the specific figures are not claimed to generalise to AndroZoo or to more recent samples. No cross-dataset validation was performed, and the baselines are LAMD-*style* models on a different dataset with a larger drift gap, so no claim of outperforming LAMD is made anywhere in this work; the cost comparison is an equivalence computed from measured token counts, not a controlled head-to-head.

Conclusion validity. Gains are reported with paired bootstrap intervals and exact McNemar tests rather than point estimates alone, which is what exposed the Test2 null result. Statistical significance is not practical importance: a +2.30 F1 gain at $p = 0.0167$ is real but modest, and the adversarial evaluation shows that a statistically sound improvement can coexist with a security property that does not hold at all.

6.7 Per-family behaviour

Aggregate F1 conceals which malware the cascade actually recovers. Examining the escalated set by family shows the gate concentrates on families whose artifact profile has shifted most from the training period, which is the intended behaviour, but also that a residual group of families is missed by both stages. Because the override band is confined to the probability region where the detector is unreliable, malware the detector confidently misclassifies as benign never reaches the reasoner at all — it falls outside $\mathcal{B}$ by construction. That is a structural limit of banded override rather than a tuning problem, and it explains why the gain saturates at an override rate of 3.4% of inputs regardless of how much wider the gate opens. Widening the band would recover some of those samples at the cost of re-admitting the blind-override failure documented above, so the two cannot be improved independently.

7 Discussion

What the evidence supports. RQ1 and RQ2 are answered cleanly: detectors degrade substantially under temporal drift, and margin-based routing concentrates errors roughly thirteenfold where the field's default conformal signal does not exceed chance. RQ4 is answered affirmatively — faithfulness can be measured without human ground truth,

and the closed vocabulary is what makes it possible. RQ3 is answered only *partially*: the cascade improves two of three windows and is statistically indistinguishable from the detector on the third, so the claim that the LLM stage improves the final decision holds conditionally, not universally. RQ5 is answered, but its robustness component is negative.

Selective versus uniform escalation. Routing is not merely cheaper than sending every application to the LLM; it is *better*. Uniform acceptance degrades F1 by 15.56 points on the hardest window, and the LLM-only operating point does not significantly outperform the cheap detector in any window. The reasoning stage earns its cost on an identifiable minority of inputs, and spending it uniformly dilutes that advantage until it disappears — which is precisely the regime TraceRAG and LAMD operate in.

Conformal validity under drift. The predictor is near-exact in-period (4.97% error at nominal 0.05) and degrades to 26–47% under drift. This is expected, since exchangeability is what drift violates, and it must not be glossed: calibration on in-period data does *not* license a validity claim under severe drift. That is why conformal machinery is confined here to abstention rather than ranking.

Faithfulness is not correctness. This is the most important interpretive point in the work. A grounded explanation can support a wrong verdict, and under adversarial feature addition the two properties were anti-correlated: mean grounding was 1.000 on samples where both stages were fooled, higher than the 0.782 observed when the reasoner resisted. EGR, CFN and AA are **integrity** measures, establishing that an explanation refers to artifacts the application genuinely contains — exactly what prior systems cannot check. They are not correctness measures and carry no adversarial guarantee. A deployment reading high grounding as a licence to trust a verdict would be misusing them.

Limits. Statistical significance is not practical importance: a +2.30 F1 gain at $p = 0.0167$ is real but modest. Temporal evaluation on one dataset is not real-world validity; KronoDroid ends in 2020, timestamps are attacker-forgeable proxies for first-seen date, and no cross-dataset validation was performed. AA is measured against SHAP, itself only a model of importance, and Galadima et al. (2026) show explainers disagree. Finally, analysis is feature-level rather than code-level, so the reasoner cannot discover behaviour the extractor did not capture; the native-library blind spot Zhang et al. (2025) identify is not resolved here. None of these results should be read as deployment readiness.

8 Conclusion and Future Work

This work asked whether conformal-calibrated selective escalation to an LLM reasoner could preserve detection quality under concept drift while reducing LLM usage and measuring explanation faithfulness without human ground truth. The answer is a qualified yes: faithfulness can be measured automatically and LLM usage cut to roughly a tenth of inputs, but the detection gain holds on two of three drift windows rather than universally.

Against the objectives, **O1** is met — conventional detectors degrade sharply, one collapsing from 76.98 to 17.34 F1. **O2** is met, with margin-based routing concentrating errors 13.07–14.42× where conformal novelty reached only 0.47–1.37. **O3** is partially met: selective escalation gains +6.51 ($p = 0.0001$) and +2.30 ($p = 0.0167$) on two windows and is not significant on the third, while uniform acceptance degrades F1 by 15.56. **O4** is met, with three annotation-free metrics whose viability rests on the closed vocabulary, removal of which collapses grounding from 0.991 to 0.000. **O5** is met, and its adversarial

component returns the study's most important negative finding.

The main technical contribution is an automatic, human-label-free faithfulness audit made possible by constraining the reasoner to a closed artifact vocabulary. The main negative finding is that grounding and correctness are not merely distinct but can be *anti-correlated* under attack, mean grounding reaching 1.000 on wrong verdicts. The practical recommendation follows directly: treat grounding as an integrity filter that decides whether an explanation is fit to show an analyst, never as evidence that a verdict is right, and separate the decision to consult a reasoner from the decision to adopt its verdict.

Limitations are stated in Section 7: a single dataset ending in 2020, feature-level rather than code-level analysis, forgeable timestamps, and metrics that measure integrity rather than correctness. Future work follows from them. More temporal windows and multiple Android datasets would test whether the protocol transfers; expanding dynamic analysis and extending the closed vocabulary to decompiled method signatures would recover code-level detail without losing verifiability; drift-aware or learned routing could replace the fixed gate, trained on the directly supervised signal the routing diagnostic already produces; human analyst validation would establish whether high-grounding explanations are genuinely more useful in triage; stronger adversarial threat models and a defence that weights citations by how discriminative an artifact is would address the attack demonstrated here; separating faithfulness from correctness deserves a metric of its own; and evaluation across more open-source LLMs, with cost-aware deployment studies, would establish where the operating point should sit in practice.

)

References

Alhogail, A. and Alharbi, R. A. (2025). Effective ML-based Android malware detection and categorization, *Electronics* **14**: 1486.

Arp, D., Spreitzenbarth, M., Hübner, M., Gascon, H. and Rieck, K. (2014). DREBIN: Effective and explainable detection of Android malware in your pocket, *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.

Augello, A., Paola, A. D. and Re, G. L. (2025). Hybrid multilevel detection of mobile devices malware under concept drift, *Journal of Network and Systems Management* **33**.

Barbero, F., Pendlebury, F., Pierazzi, F. and Cavallaro, L. (2022). Transcending TRANSCEND: Revisiting malware classification in the presence of concept drift, *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 805–823.

Fayyazi, R., Taghdimi, R. and Yang, S. J. (2024). Advancing TTP analysis: Harnessing the power of large language models with retrieval augmented generation, *2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*, pp. 255–261.

Feng, R., Chen, H., Wang, S., Karim, M. M. and Jiang, Q. (2025). LLM-MalDetect: A large language model-based method for Android malware detection, *IEEE Access* **13**: 81347–81364.

Galadima, K., Anthony, P., Ploszek, R., Balogh, Š., Zajac, P. and Homola, M. (2026). Evaluating the trustworthiness of LEMNA versus LIME and SHAP in explainable malware detection, *Proceedings of the 18th International Conference on Agents and Artificial Intelligence (ICAART)*, pp. 3353–3364.

Guerra-Manzanares, A., Bahşi, H. and Nõmm, S. (2021). KronoDroid: Time-based hybrid-featured dataset for effective Android malware detection and characterization, *Computers & Security* **110**: 102399.

He, Y., Lei, J., Qin, Z., Ren, K. and Chen, C. (2025). Combating concept drift with explanatory detection and adaptation for Android malware classification, *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 978–992.

I, M., Shankar, A., V, V., Gopalakrishnan, S. and Vaddina, V. (2024). Mitigating factual inconsistency and hallucination in large language models, *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*, pp. 1169–1170.

Jing, C., Wu, Y. and Cui, C. (2026). SaBER-LLMs: Security-aware behavior embedding representation via large language models for fine-grained malware analysis, *Neurocomputing* **680**: 133234.

Kwon, N., Roh, K., Hwang, Y., je Cho, S. and Kang, B. (2026). C-STAR: Cost-aware adaptive learning under concept drift for Android malware detection, *Proceedings of the 23rd International Conference on Security and Cryptography (SECRYPT)*, pp. 817–824.

Lan, T. and Nait-Abdesselam, F. (2025). LAMLAD: LLM-based adversarial attack against machine learning for Android malware detection, *2025 IEEE Conference on Communications and Network Security (CNS)*, pp. 1–9.

Li, A. S., Iyengar, A., Kundu, A. and Bertino, E. (2025). Revisiting concept drift in Windows malware detection: Adaptation to real drifted malware with minimal samples, *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS)*.

Manthena, H., Shajarian, S., Kimmell, J. C., Abdelsalam, M., Khorsandroo, S. and Gupta, M. (2025). Explainable artificial intelligence (XAI) for malware analysis: A survey of techniques, applications, and open challenges, *IEEE Access* **13**: 61611–61640.

Minnei, L., Eddoubi, H., Sotgiu, A., Pintor, M., Demontis, A. and Biggio, B. (2024). Data drift in Android malware detection, *2024 International Conference on Machine Learning and Cybernetics (ICMLC)*, pp. 157–163.

Nadeem, A., Vos, D., Cao, C., Pajola, L., Dieck, S., Baumgartner, R. and Verwer, S. (2023). SoK: Explainable machine learning for computer security applications, *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, pp. 221–240.

Najibi, M. and Bidgoly, A. J. (2025). Towards a robust Android malware detection model using explainable deep learning, *Journal of Information Security and Applications* **93**: 104191.

Noppel, M. and Wressnegger, C. (2024). SoK: Explainable machine learning in adversarial environments, *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 2441–2459.

Qian, X., Zheng, X., He, Y., Yang, S. and Cavallaro, L. (2025). LAMD: Context-driven Android malware detection and classification with LLMs, *2025 IEEE Security and Privacy Workshops (SPW)*, pp. 126–136.

Rafiq, H., Aslam, N., Issac, B. and Randhawa, R. H. (2022). On impact of adversarial evasion attacks on ML-based Android malware classifier trained on hybrid features, *2022 14th International Conference on Software, Knowledge, Information Management and Applications (SKIMA)*, pp. 216–221.

Rashid, M. U., Qureshi, S., Abid, A., Alqahtany, S. S., Alqazzaz, A., ul Hassan, M., Reshan, M. S. A. and Shaikh, A. (2025). Hybrid Android malware detection and classification using deep neural networks, *International Journal of Computational Intelligence Systems* **18**.

Rollinson, N. and Polatidis, N. (2026). LLM-generated samples for Android malware detection, *Digital* **6**: 5.

Sabbah, A., Jarrar, R., Zein, S. and Mohaisen, D. (2025). Empirical evaluation of concept drift in ML-based Android malware detection. arXiv preprint arXiv:2507.22772 (not peer reviewed).

Saeed, N. H., Hamza, A. A., Sobh, M. and Bahaa-Eldin, A. M. (2026). Efficient feature ranked hybrid framework for Android IoT malware detection, *Scientific Reports* **16**.

Sakthidharan, S., Anagha, S. M., Arikkat, D. R., Rehiman, K. A. R., Nicolazzo, S., Nocera, A. and Vinod, P. (2026). Enhancing Android malware detection with retrieval-augmented generation, *International Journal of Information Security* **25**.

Saqib, M., Mahdavifar, S., Fung, B. C. M. and Charland, P. (2024). A comprehensive analysis of explainable AI for malware hunting, *ACM Computing Surveys* **56**: 1–40.

Shukla, S. and Joshi, H. (2025). Proof-carrying answers: A systematic protocol for verifiable retrieval-augmented generation with cryptographic provenance, *2025 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*, pp. 405–413.

Singh, A., Tanha, M., Girdhar, Y. and Hunter, A. (2024). Interpretable Android malware detection based on dynamic analysis, *Proceedings of the 10th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 195–202.

Zhang, G., Li, H., Chen, Z., Peng, L., Zhu, Y. and Zhao, C. (2021). AndroCreme: Unseen Android malware detection based on inductive conformal learning, *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pp. 651–658.

Zhang, G., Wang, X., Sun, S., Xiao, P., Sun, K. and Xiong, Y. (2025). TraceRAG: A LLM-based framework for explainable Android malware detection and behavior analysis. arXiv preprint arXiv:2509.08865 (not peer reviewed).

Zhao, W., Wu, J. and Meng, Z. (2025). AppPoet: Large language model based Android malware detection via multi-view prompt engineering, *Expert Systems with Applications* **262**: 125546.

Zheng, X., Qian, X., He, Y., Yang, S. and Cavallaro, L. (2025). Is "knowing it's malicious" enough? evaluating LLMs for fine-grained malware behavior auditing. arXiv preprint arXiv:2509.14335 (not peer reviewed).